

Agile Hardware and Software Co-design for RISC-V-based Multi-precision Deep Learning Microprocessor

Zicheng He^{*†}, Ao Shen^{*}, Qiufeng Li^{*}, Quan Cheng^{*}, Hao Yu^{*}

Southern University of Science and Technology^{*} University of California, Los Angeles[†]
yuh3@sustech.edu.cn

ABSTRACT

Recent network architecture search (NAS) has been widely applied to simplify deep learning neural networks, which typically result in a multi-precision network. Many multi-precision accelerators have been developed as well to support computing multi-precision networks manually. A software-hardware interface is thereby needed to automatically map multi-precision networks onto multi-precision accelerators. In this paper, we have developed an agile hardware and software co-design for RISC-V-based multi-precision deep learning microprocessor. We have designed custom RISC-V instructions with a framework to automatically compile multi-precision CNN networks onto multi-precision CNN accelerators, demonstrated on FPGA. Experiments show that with NAS optimized multi-precision CNN models (LeNet, VGG16, ResNet, MobileNet), the RISC-V core with multi-precision accelerators can reach the highest throughput in 2,4,8-bit precisions respectively on a Xilinx ZCU102 FPGA.

ACM Reference Format:

Zicheng He^{*†}, Ao Shen^{*}, Qiufeng Li^{*}, Quan Cheng^{*}, Hao Yu^{*}. 2023. Agile Hardware and Software Co-design for RISC-V-based Multi-precision Deep Learning Microprocessor. In *Proceedings of ACM Asia South Pacific Design Automation Conference (ASP-DAC 2023)*. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3566097.3567871>

1 INTRODUCTION

NAS-based quantization is proven to achieve a well trade-off between network performance and hardware efficiency [1]. To satisfy the demand of acquired multi-precision network, accelerators targeting specific workloads are always adopted, which however demand high engineering effort to support new operators for other networks. Heterogeneous systems [2, 3] coupling accelerator with a core can delegate accelerator-unsupported kernels to the core therefore exhibiting excellent flexibility for various deep learning applications. The model inference on such systems still has to be completed with manual deployment to achieve the full utilization of hardware primitives for acceleration, severely hindering its utility for a wide diversity of deep learning workloads. Recent advances in deep learning compilers [4–6] have provided great potential for automatic model deployment. There are however few frameworks

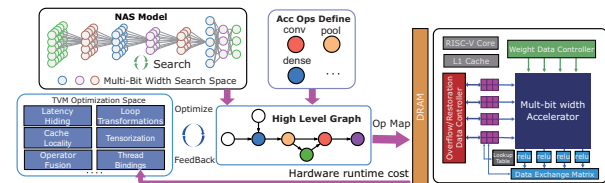


Figure 1: Co-design for RISC-V-based multi-precision deep learning microprocessor and deep learning framework.

developed supporting model deployment with mixed precision quantization on embedded devices.

By bringing heterogeneous systems together with the deep learning frameworks, we present a hardware/software co-design microprocessor for a wide variety of deep learning applications, whose framework is shown in Figure 1. At the hardware level, we adopt the architecture design of bit-split-and-combination (BSC) accelerator on multi-precision operation, then integrate the accelerator with a RISC-V core by our custom designed instructions to fit the computation need of accelerator-unsupported kernels. At the software level, we complement TVM [7] stack and develop our custom deep learning framework for kernel function generation and inference formulation. To achieve maximized inference speedup, graph-level and operator-level optimizations are both carefully employed during compilation.

The main contribution of this work comes from the agile hardware/software co-design topology we have taken for our multi-precision deep-learning microprocessor. The hardware we present can support nowadays major CNN networks with multi-precision quantization and can be easily extended for future deep learning models. Furthermore, model deployment on the proposed microprocessor can be automated to achieve competitive overall performance by complementing TVM stack with multi-precision quantization support and custom RISC-V instructions.

Experiments demonstrated on ZCU102 board show that with NAS optimized multi-precision CNN models (LeNet5, VGG16, ResNet, MobileNet), our microprocessor can reach 1633 GOPS, 810 GOPS and 429 GOPS throughput in 2,4,8-bit precision convolution respectively with automatic model deployment by fully utilizing the hardware accelerator for 2-8 bit operation mapping.

The rest of the paper is organized as follows. Section 2 describes the hardware design for the mixed-precision accelerator and corresponding SoC. Section 3 presents the multi-precision compiler for the designed hardware. Section 4 describes the multi-precision system. Section 5 shows our experimental results on CNNs. Section 6 concludes the whole work.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASP-DAC 2023, January 16–19, 2023, Tokyo, Japan

© 2023 Association for Computing Machinery.

ACM ISBN 978-1-4503-9783-4/23/01...\$15.00

<https://doi.org/10.1145/3566097.3567871>

2 MULTI-PRECISION ACCELERATOR

2.1 Multi-precision Network Model

Our multi-precision model definition is shown in Figure 2. The search space $\mathcal{A}$ includes three bit-width variations of all dense and convolution operators in the model. As the possible bit-width of these operators could be quite low, we employ quantization-aware training (QAT) to maintain a reasonable accuracy loss during search. However, it is difficult to directly derive the optimal bit width through the differential training method by calculating $\frac{\partial L}{\partial b}$, $b \in \text{bit width}$, we design a new method by mapping the layer attributes, 2-8 bit-width here, to the differential architecture parameter α . To avoid the influence of the α itself on the accuracy of the model, which cannot reflect the proportion of different branches, gumbel-softmax (also named softmax^G) is introduced in the training for normalization. The $\hat{\alpha}$ is used as the branch proportion in the searching:

$$\hat{\alpha} = \text{softmax}^G(\alpha) \quad (1)$$

When the search training of the model is completed, the optimal

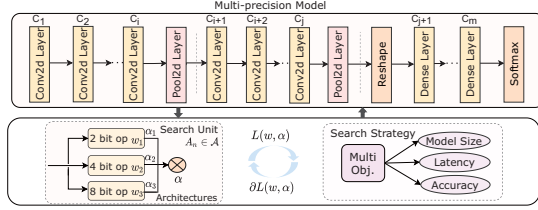


Figure 2: Multi-precision quantization method

quantization bit width of the multi-precision operator can be transformed into minimizing the loss function related to α as below:

$$\min_{A \in \mathcal{A}} \min_{W, \alpha} L(W, \alpha) \quad (2)$$

After the whole training is completed, in the multi-precision model, each multi-precision operator will have corresponding precision information, and the compiler could map the operator to the hardware according to its precision information.

2.2 Multi-precision Accelerator

A re-configurable BSC systolic CNN accelerator inspired by [8–10] is proposed as the computing unit to support multi-precision operations derived from QAT. This multi-precision accelerator is based on 4-bit computation and supports multi-precision multiply-and-accumulate (MAC), including 8-bit, 4-bit, and 2-bit. Compared with ordinary MAC math functions, multi-precision MAC requires two more loops to combine the results to obtain different precision results.

$$f_{calc}(x, y, z) = \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} \sum_{m=1}^{C_{in}} F_{i,a}(x+i, y+j, m) K_{M,b}(i, j, m, z) \quad (3)$$

$$F_{out}(x, y, z) = \sum_{a=0}^{U-1} (2^a \cdot \sum_{b=0}^{U-1} (2^b \cdot f_{calc}(x, y, z)))$$

Based on such multi-precision MAC mathematical foundation, we implement our multi-bit-width systolic accelerator whose structure is shown in Figure 3. In 8-bit model, an 8-bit input activation and an 8-bit weight are both divided into two 4-bit numbers. Then

the 8 bit×8 bit multiplication is decomposed into four 4 bit×4 bit multiplications correspondingly. In order to reduce the logic overhead of shifting operations, the 4 bit×4 bit products perform fixed leftward shift operations by parameters of 0, 2, 2, and 4 respectively. When calculating 4 bit×4 bit precision case in 4-bit model, four 4 bit×4 bit multiplications are performed in parallel and added up together without shifting. For 2 bit×2 bit precision case in the 2-bit model, 8 groups of 2 bit×2 bit multiplications are performed in parallel and added up together with specific shifters. This BSC method can support 9 multiplication cases (2 bit, 4 bit, and 8 bit for both input activation and weight). For different multiplication cases, the shifter for the products of 4 bit×4 bit are selectively performed by shifting parameters of 0, 4 and 8 according to the computation precision.

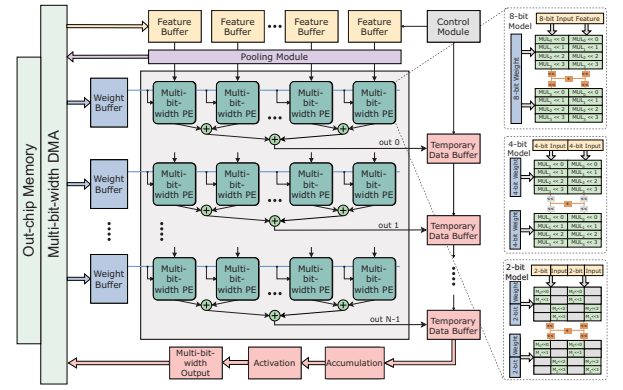


Figure 3: Hardware architecture of multi-precision accelerator

We also add a pooling operation unit into this accelerator to support pooling operator. Moreover, to integrate this accelerator into our hardware system, we deploy an AXI4-Lite bus into the accelerator for control. To fetch the input and weight data quickly, an AXI4-Full bus is employed on the platform while a DMA module for automatic data access. In short, this multi-precision accelerator connected with a RISC-V core is regarded as a peripheral with DMA channel in our hardware system.

3 MULTI-PRECISION COMPILER

In this section, we discuss how to use a compiler inheriting existing Deep Learning compiler infrastructure to automate operator generation for diverse network models on the designed hardware. We reformulate the computation graph to categorize operators into two classes and apply different optimization methods to each type of operator. The details of each compiler-specific steps are discussed as follows.

3.1 Multi-precision Computation Graph

The design of our whole system necessitates a heterogeneous computing pattern: for those unsupported kernels, we should keep them on the RISC-V core while multi-precision operators are delegated to the accelerator for computation. The computation graph is expected to be compatible with both types of operators and should reflect

their differences. To derive the required computation graph automatically, we use the TVM front-end to read the multi-precision model and transform it into Relay[11] IR. Then *Graph Transformation* including three stages, is applied to the IR module obtained shown in Figure 4.

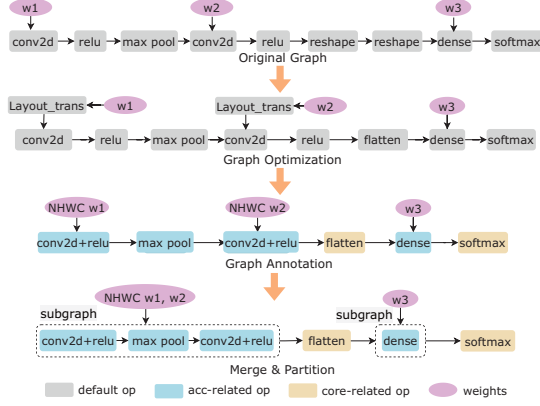


Figure 4: Graph transformation stages

Graph Optimization is first conducted with the help of TVM stack so that a large portion of existing compiler passes can be reused for high-level dataflow rewriting. The existing target-independent passes we take during the optimization include constant folding and data-layout transformation for execution cost saving. Operator fusion is also applied to the computation graph with awareness of our hardware architecture and will be conducted if the fused operator is supported by our hardware.

Graph Annotation takes the optimized graph as input and manages to distinguish accelerator-related kernels from other operators. By registering all the accelerator supported kernels on the whitelist, every operator in the computation graph will be checked by going through the list. The registered kernels will be explicitly denoted as the external functions for further processing.

We employ *Merge & Partition* to group adjacent accelerator-related kernels together to form subgraphs that can be entirely offloaded to the accelerator. Here we greedily merge as many accelerator-enabled kernels as possible while guaranteeing the functional correctness. This means less synchronization, minimized data transaction, and better performance. For each of the subgraphs from this stage, we create a Relay function inheriting the external attribute of accelerator-related kernels that is dispatched to our FPGA backend for code generation.

3.2 Accelerator-related Operator Mapping

Given the computation graph, it cannot guarantee the efficient model inference on the whole system unless operator computation is optimized. We now discuss the code generation for accelerator-related operators. As adjacent accelerator-related operators are combined as a subgraph for code generation, the parsing of the subgraph is first introduced. The main idea is to implement a Relay graph visitor iteratively visiting each node in it. We examine the operation containing in the node and translate it into a corresponding hardware operation API. During the translation of each operation,

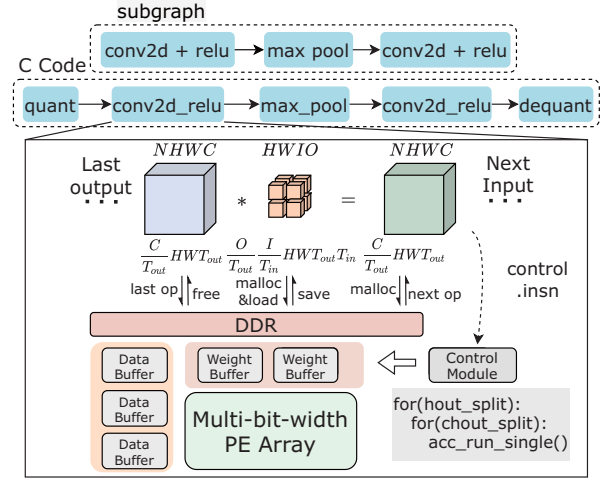


Figure 5: Multi-Precision Operator Mapping Accelerator

the compiler derives the attributes of the operation from the node then hard code them as function arguments to correctly invoke the interface. The tensor sizes of inputs and outputs are used to calculate the dynamic memory allocation for intermediate results and the address pointers of these buffers are traced to instruct the data movement between the core, the DRAM and the accelerator. Considering the limited memory space on the microprocessor, the dynamic memory will be immediately released if it is not needed by any proceeding operations in the subgraph. After that, the C snippet of the subgraph can be generated as shown in Figure 5.

The snippet of the subgraph is an incomplete program as the hardware operations listed do not yet have concrete implementation. For those functions, computation and communication should be considered at the meantime for efficient execution. This is done through manual effort: We first develop several intrinsic functions that will be turned into our custom instructions during compilation to read and write the accelerator registers efficiently. Then we provide a hand-tuned, well-tiled, computation-memory balanced software library that implements those operation based on the intrinsic functions. This approach facilitates fast hardware/software co-design and ensures the efficient operation computation on the accelerator.

3.3 Core-related Operator Mapping

The manual approach we take for accelerator-related operator is not sustainable for designing unsupported kernels considering the wide diversity of them. To generate efficient code for such operators, Halide’s [6] idea of separating computation and scheduling is adopted and *Tensor Schedule Optimization* is applied with the help of Auto-scheduler as known as Ansor [4] to find the optimal schedule on our RISC-V core. We now discuss how to adopt such a framework to enable efficient tensor program generation and evaluation for our co-design.

As the accelerator-related kernels are delegated to hand-tuned libraries for optimization, we only need to explore the scheduling space of core-related operators. To cut off redundant space, We here

use the similar pattern recognition method mentioned in Section 3.1 and maintain the scheduling tasks of core-related operators. To get the performance of an operator with specific scheduling on our microprocess without TVM runtime support, we develop our interface by using OpenOCD to flash the generated tensor program with correct arguments to the hardware and fetch back profiling results. Considering the limited bandwidth between the host and device, the interface will delegates random assignment of program input to hardware itself instead of transmitting from host. After deriving the runtime cost of operator program with specific scheduling, the further choice of operator candidates can be guided to iteratively reach the optimal scheduling for core-related operators.

4 MULTI-PRECISION SYSTEM

4.1 RISC-V based Multi-precision Microprocessor

To provide flexibility and generality for DL workloads, our system design couples the multi-precision accelerator with a core. The entire block diagram is shown in Figure 6. The core is an open-source 32-bit RISC-V core called E203 with a two-stage pipeline based ultra-low power/area implementation, which also provides a complete development environment for the convenience of usage. Based on that, we enable a wealth of peripherals like UART, JTAG, Flash, etc., on our platform.

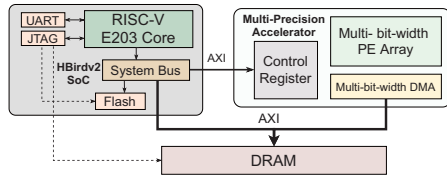


Figure 6: Block diagram of RISC-V based multi-precision deep learning microprocessor

To invoke the accelerator for computation, RISC-V core has to write the configuration information of operator to parameter registers and then set the ST register. Moreover, after the computation is completed, RISC-V CPU needs to wait for the synchronization signal from the accelerator to fetch the correct result. Addressing too many serial instructions used during these behaviors, we have designed three custom R-type instructions to implement fast reading and writing of registers in Table 1. CL_RG, LD_RG, and ST_RG represent cleaning, loading, and storing data, respectively. In the custom instruction set, the register *rs1* and *rs2* store the number of accelerator registers and the value to be written. The value in accelerator registers is written to register *rd*.

Table 1: RISC-V instructions for operating accelerator registers

31:25	24:20	19:15	14	13	12	11:7	6:0
func7	rs2	rs1	xd	xs1	xs2	rd	opcode
CL_RG	00000	00000	0	0	0	00000	CUSTOM-0
LD_RG	00000	rs1	1	1	0	rd	CUSTOM-0
ST_RG	rs2	rs1	0	1	1	00000	CUSTOM-0

For the hardware implementation of custom instructions, we use the interface of the co-processor reserved by E203 to send them marked as CUSTOM-0 to the co-processor unit for execution, and the main processor can continue to process the next instruction.

Table 2: Resource allocation of proposed microprocessor

Device Module	Start Address	End Address
Debug Module	0x0000_0000	0x0000_0FFF
UART Serial Port	0x1001_3000	0x1001_3FFF
DRAM Module	0x4000_0000	0x4FFF_FFFF
Acc Control Register	0x5000_0000	0x5000_FFFF

JTAG port and OpenOCD debugger are used to connect with microprocessor and host when RISC-V ELF and parameters are loaded into the hardware. UART port is used to monitor the state of microprocessor and fetch the feedback. In all, the resource allocation for the microprocessor hardware is shown in Table 2.

4.2 Multi-precision Model Control Flow

Our compiler can parse the multi-precision model into a list of operations and produce efficient code for their implementation. However, we still need strict operator control flow in line with the definition of the multi-precision model to finish the model deployment. This is achieved through an offline analysis to produce a static C runtime. We first read the model summary file to produce the correct operator sequence and maintains a map storing the name of each operator and its tensor inputs and outputs. By calculating the tensor sizes, the memory allocation for weight parameters can be determined, and corresponding address pointers are derived. Then the dataflow between the core, the DRAM and the accelerator will be statically written in the runtime by assigning the correct pointers to the operator interface according to the map. The bit-width for each multi-precision operation is hard coded in the runtime at last. After that, the runtime is fixed for the multi-precision model and can be directly invoked for inference.

The runtime code is finally compiled with RISC-V kernels and subgraphs functions into an ELF and then offloaded to our system for inference. The execution flow is shown in Figure 7, implying that such an ahead-of-time runtime can reduce the overhead in terms of code size and inference speed, both of which are crucial for our system.

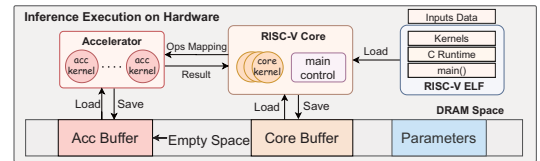


Figure 7: ELF execution flow

5 EXPERIMENTS

To evaluate the proposed hardware/software co-design, we practically implement the multi-precision microprocessor on FPGA. The hardware performance compared with the state-of-the-art FPGA accelerator and overall latency improvement through automatic deployment and auto-tuning technique are reported.

5.1 Experiment Setup

The RISC-V based multi-precision microprocessor is synthesized by Vivado 2018 and implemented on the ZCU102 board. The resource utilization on a Xilinx ZCU102 is shown in Table 3. For this multi-precision microprocessor, due to the limitation of FPGA, the working frequency of RISC-V Core (soft core) cannot be too high and we use 16 MHz to achieve. Besides, the PE array of accelerator is 32×32 and working frequency is 214 MHz. So, the theoretical throughput is 438 GOPS for 8-bit operator, 876 GOPS for 4-bit operator and 1753 GOPS for 2-bit operator.

Table 3: Resource utilization of proposed microprocessor on ZCU102

Resource	LUT	FF	BRAM	DSP
Available	274080	548160	912	2520
Used	142268	92405	289	1373
Utilization	51.91%	16.86%	31.69%	54.48%

In our multi-precision model experiment, we design a multi-objective loss function, defined as the sum of the model accuracy error and the hardware throughput simulation cost, which not only considers model size and accuracy but also introduces a runtime-related factor to realize the balance between hardware efficiency and accuracy. The optimal multi-precision model is obtained by using stochastic gradient descent with adaptive momentum optimization in 300 epochs.

5.2 Throughput Comparison

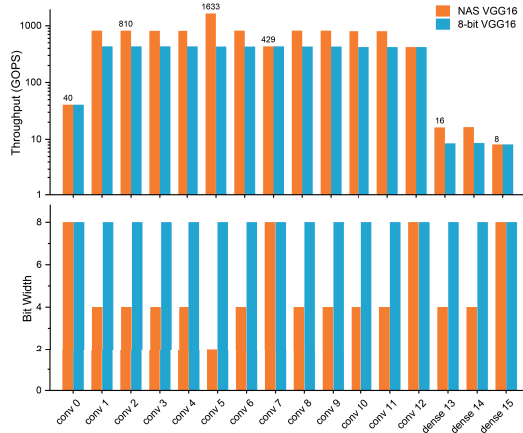


Figure 8: Throughput and bit-width of the NAS-based VGG16 and 8-bit VGG16 model at conv2d and dense layers

We first test the hardware performance with a pruned VGG16 model that can be entirely delegated to the accelerator. Compared with 8-bit model, NAS-VGG16 allows lower bit-width operators in the model so that it can achieve higher throughput at all convolutional layers as shown in Figure 8. For the multi-precision operators, the accelerator could reach 1633 GOPS for 2-bit in conv 5, 810 GOPS for 4-bit in conv 2 and 429 GOPS for 8-bit in conv 7, all of which are very close to the theoretical limit shown in Section 5.1. This result

shows that our accelerator and corresponding data flow are well tailored for the multi-precision operators. Meanwhile, it proves the minimal effect of core integration.

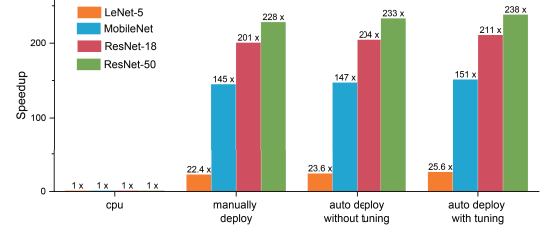


Figure 9: Latency comparison (Normalized over embedded RISC-V CPU data).

We also compare our results with prior specialized FPGA accelerators and the comparison table is shown in Table 4. As shown, the throughput of NAS-based VGG16 and 8-bit VGG16 decreases to 494.10 GOPS and 295.23 GOPS respectively, which mainly results from the pooling operators and the dense operators (with lots of weights and constrained by the Roof-Line model). Compared with previous work, our accelerator achieves more than 1.5 $\times$ improvement in normalized throughput.

We use four classic CNN classification models with NAS-based optimization: LeNet-5[17], MobileNetv1[18], ResNet-18 and ResNet-50[19] to evaluate our microprocessor and the corresponding compiler solution for model deployment. We deploy these models on our microprocessor without hardware reconfiguration, whose results are compared in Figure 9 with all latency normalized with respect to the baseline execution on RISC-V Core.

The significant variability in the performance boosts achieved by different models mainly results from workload executed on the accelerator. For example, compiler-based deployment achieves 23.6 $\times$ speedup over RISC-V Core when running LeNet-5 that contains only 3 convolutional layers and 2 dense layers while achieving 233 $\times$ speedup for ResNet-50 with much more operators supported by accelerators. The general superior performances to manual method are caused by the high-level optimization during multi-precision computation graph generation. Such performance can be improved even more from better scheduling of the core-related kernels, which only need easy set-up for Ansor and off-line design space exploration.

5.3 Accuracy Comparison

We show the comparison of VGG-16 using different precision schemes in Table 5. Compared to the float model, the multi-precision model has an accuracy loss within 1.0% as the 8-bit model does while it can compress the model by 7.3 $\times$, much more than 8-bit model does.

The training results of four CNN-based NAS-optimized multi-precision models is shown in Table 6. It exhibits the similar results as VGG-16 study case where the loss of model accuracy is acceptable when considering the accelerator calculation throughput and accuracy at the same time. As the 4-bit calculation-based search results account for the majority, the final size of the multi-precision model has a compression rate of basically 7 $\times$, while the 8-bit model can only be compressed by 4 $\times$.

Table 4: Accelerator performance comparison

	Device	Network	Accuracy ¹ (Top-1)	Input Precision	Weight Precision	LUTs (Ratio)	DSP (Ratio)	Freq. (MHz)	Through. (GOPS)
[12]	XC7Z045	VGG16	66.96%	8	8	182.6K(84%)	780(87%)	150	188
[13]	XC7Z045	VGG16	68.02%	16	16	183K(84%)	780(89%)	150	137
[14]	VX690t	VGG16	66.52%	16	16	-	-	150	203.9
[15]	GX1150	VGG16	-	16	8	161K(14%)	1518(100%)	150	645.25
[16]	VX690t	VGG16	-	16	8	3E5(81%)	2833(78%)	150	354
ours	ZCU102	VGG16	71.3%	8	8	142.3K(51.9%)	1373(54.5%)	214	295.23
ours	ZCU102	NAS-VGG16	70.9%	multi	multi	141.8K(51.7%)	1373(54.5%)	214	494.10
ours	ZCU102	single conv op	-	8	8	142.3K(51.9%)	1373(54.5%)	214	429.25
ours	ZCU102	single conv op	-	4	4	142.3K(51.9%)	1373(54.5%)	214	810.54
ours	ZCU102	single conv op	-	2	2	142.3K(51.9%)	1373(54.5%)	214	1633.07

¹ The dataset using ImageNet.**Table 5: Results of VGG16 models with different precision schemes on ImageNet**

	Precision	Accuracy (Top-1)	Weight (Ratio)
VGG16	float	71.9%	513MB
	8-bit	71.3%	128MB(25.0%)
	multi	70.9%	70MB(13.6%)

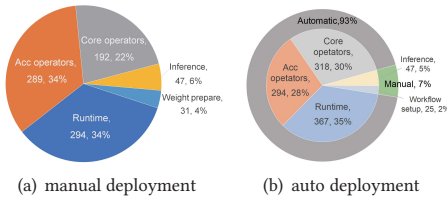
Table 6: Quantization results of CNN models on CIFAR10

Model	Top-1 accuracy ¹	8/4/2 bit ratio	Weight
LeNet-5 ²	96.1% (0.6%)	60%/20%/20%	23KB
MobileNet	90.1% (4.8%)	4.8%/77.9%/17.3%	2.87MB
ResNet-18	94.0% (1.2%)	3.7%/78.3%/18.0%	7.93MB
ResNet-50	94.1% (2.0%)	0.6%/78.3%/21.1%	15.16MB

¹ Data in brackets is the accuracy loss of multi-precision models compared with float models.² LeNet-5 results are derived on MNIST.

5.4 Agility Comparison

For a more quantitative analysis in the development cost saving, we present the Line-of-Code needed for MobileNet deployment as a case study shown in Figure 10. Our method only requires users to call some APIs to set-up the compiler workflow and some code to invoke the compiled model for image classification while manual method demands hundreds of lines of code and imposes a heavy burden on both the budget and the timetable of a project. Such advantages of our method facilitates the agile development for a wide range of deep learning applications.

**Figure 10: Line-of-Code comparison for MobileNet**

6 CONCLUSION

We proposed an agile hardware/software co-design microprocessor featured with automatic NAS-based optimized models deployment. We implemented the heterogeneous architecture using BSC accelerator controlled by custom RISC-V instructions, which exhibits high flexibility and decent performance for various CNN models. We also developed our compiler with custom FPGA backend integration to automate DL workloads mapping with both graph-level

and operator-level optimizations. We hope this can open the possibilities of agile development of deep learning microprocessor.

ACKNOWLEDGEMENT

This work was supported by the National Key R&D Program of the Ministry of science and technology under Grant 2021YFE0204000, in part, in part by the National Natural Science Foundation of China (NSFC) Key Program under Grant 62034007, in part by the Guangdong Provincial Key Laboratory Program from the Department of Science and Technology of Guangdong Province under Grant 2021B1212040001, and in part by the Shenzhen Science and Technology Program under Grant KQTD20200820113051096.

REFERENCES

- [1] B. Wu *et al.*, “Mixed quantization of convnets via differentiable neural architecture search,” *ArXiv preprint arXiv:1812.00090*, 2018.
- [2] P. N. Whatmough *et al.*, “14.3 a 28nm soc with a 1.2ghz 568nj/prediction sparse deep-neural-network engine with >0.1 timing error rate tolerance for iot applications,” *JSSCC*, pp. 242–243, 2017.
- [3] F. Farshchi *et al.*, “Integrating nvidia deep learning accelerator (nvla) with risc-v soc on firesim,” *EMC2*, pp. 21–25, 2019.
- [4] L. Z. others, “Ansor : Generating high-performance tensor programs for deep learning,” in *OSDI*, 2020.
- [5] S. Zheng *et al.*, “Flextensor: An automatic schedule exploration and optimization framework for tensor computation on heterogeneous system,” *ASPLOS*, 2020.
- [6] Ragan-Kelley *et al.*, “Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines,” *SIGPLAN Not.*, vol. 48, no. 6, p. 519–530, 2013.
- [7] T. Chen *et al.*, “Tvm: An automated end-to-end optimizing compiler for deep learning,” in *OSDI*, 2018.
- [8] L. Dai *et al.*, “An energy-efficient bit-split-and-combination systolic accelerator for nas-based multi-precision convolution neural networks,” in *ASP-DAC*, 2022.
- [9] M. Huang *et al.*, “An energy-efficient bit-split-and-combination systolic accelerator for nas-based multi-precision convolution neural networks,” in *FPGA*, 2022.
- [10] H. Sharma *et al.*, “Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural network,” in *ISCA*, pp. 764–775, 2018.
- [11] J. Roesch *et al.*, “Relay: A high-level ir for deep learning,” *ArXiv preprint arXiv:1904.08368*, 2019.
- [12] K. Guo *et al.*, “Angel-eye: A complete design flow for mapping cnn onto customized hardware,” in *ISVLSI*, pp. 24–29, 2016.
- [13] J. Qiu *et al.*, “Going deeper with embedded fpga platform for convolutional neural network,” in *FPGA*, *FPGA '16*, p. 26–35, 2016.
- [14] C. Zhang *et al.*, “Energy-efficient cnn implementation on a deeply pipelined fpga cluster,” in *ISLPED*, *ISLPED '16*, p. 326–331, 2016.
- [15] Y. Ma *et al.*, “Optimizing loop operation and dataflow in fpga acceleration of deep convolutional neural networks,” *FPGA '17*, (New York, NY, USA), p. 45–54, 2017.
- [16] C. Zhang *et al.*, “Caffeine: Toward uniformed representation and acceleration for deep convolutional neural networks,” *TCAD*, vol. 38, no. 11, pp. 2072–2085, 2019.
- [17] Y. Lecun *et al.*, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [18] A. G. Howard *et al.*, “Mobilenets: Efficient convolutional neural networks for mobile vision applications,” *ArXiv preprint arXiv:1704.04861*, 2017.
- [19] K. He *et al.*, “Deep residual learning for image recognition,” *CVPR '16*, pp. 770–778, 2016.